

Программирование на языке Python

Словари

Словари

Данные можно хранить в списках. Например, есть список английских слов, которые кто-то хочет выучить:

```
english_words = ['hand', 'leg',  
                 'back-end developer']
```

Удобнее было бы хранить переводы слов с русского на английский, чтобы забытые слова было легко подсмотреть.

Для этого в Python есть структура данных **dict**.



Словарь — неупорядоченная структура данных, которая позволяет хранить пары «ключ — значение»

Словари

Структура данных **dict**:

```
english = {  
    'рука': 'hand',  
    'нога': 'leg',  
    'бэкенд-разработчик': 'back-end  
    developer'  
}
```

КЛЮЧ

ЗНАЧЕНИЕ

Словарь оформляется фигурными скобками.

Его заполняют пары, записанные через запятую. Первый элемент в паре называется **ключ**, а второй — **значение**, они разделяются между собой двоеточием. Русские слова здесь ключи, а их переводы на английский — значения.

Создание словаря

Пустой словарь можно создать при помощи функции `dict()` или пустой пары фигурных скобок `{}`

Для создания словаря с некоторым набором начальных значений можно использовать следующие конструкции:

```
1) Capitals = {'Russia': 'Moscow',  
              'Ukraine': 'Kiev', 'USA':  
              'Washington'}
```

```
2) Capitals = dict(Russia = 'Moscow',  
                  Ukraine = 'Kiev', USA = 'Washington')
```

Эти два способа можно использовать только для создания небольших словарей, перечисляя все их элементы. Кроме того, во втором способе ключи передаются как именованные параметры функции `dict`, поэтому в этом случае ключи могут быть только строками

Создание словаря

```
3) Capitals = dict([("Russia", "Moscow"),  
                  ("Ukraine", "Kiev"), ("USA",  
                  "Washington")])
```

```
4) Capitals = dict(zip(["Russia",  
                       "Ukraine", "USA"], ["Moscow", "Kiev",  
                       "Washington"]))
```

В третьем и четвертом случае можно создавать большие словари, если в качестве аргументов передавать уже готовые списки, которые могут быть получены не обязательно перечислением всех элементов, а любым другим способом построены по ходу исполнения программы.

В третьем способе функции **dict** нужно передать список, каждый элемент которого является кортежем из двух элементов: ключа и значения. В четвертом способе используется функция **zip**, которой передаются два списка одинаковой длины: список ключей и список значений.

Словари

Когда запрашивают в словаре значение, соответствующее определённому ключу, это называется «доступ по ключу». Так можно получить значение для какого-нибудь ключа и заменить его:

```
english = {  
    'рука': 'hand',  
    'нога': 'leg',  
    'бэкенд-разработчик': 'back-end developer'  
}  
# доступ по ключу: как по-английски рука?  
print(english['рука'])  
# будет напечатано слово hand  
english['рука'] = 'arm'  
# значение для ключа 'рука' поменялось с 'hand' на  
# 'arm' - другой допустимый перевод
```

Словари

Ключи в словаре похожи на индексы списков. Только индексами выступают натуральные числа, а ключами бывают и числа обоих типов, и строки, и даже булевы значения True и False:

```
garden = {  
    'земляника': 'ягода',  
    'яблоко': 'фрукт',  
    'лук': ['овощ', 'оружие']  
    # значением может быть список  
}  
print(garden['лук'])  
# будет напечатано ['овощ', 'оружие']
```

Словари



Ключом список быть
не может!

```
garden = {  
    'земляника': 'ягода',  
    'яблоко': 'фрукт',  
    ['морковь', 'кабачок']: 'овощ' # так нельзя!  
}  
print(garden[['морковь', 'кабачок']])  
# получится ошибка
```

Методы словарей

Чтобы получить **все ключи** словаря, нужно вызвать метод `keys()`

```
# Получим ключи словаря:
favorite_songs = {
    'Тополиный пух': 'Иванушки international',
    'Город золотой': 'Аквариум',
    'Звезда по имени Солнце': 'Кино'
}
print(favorite_songs.keys())
# будет напечатан список песен
# dict_keys(['Тополиный пух', 'Город золотой',
# 'Звезда по имени Солнце'])
```

Методы словарей

Чтобы получить **все значения** словаря, нужно вызвать метод **values()**:

```
# Получим значения словаря
old_letters = {
    'ять': 'Ѣ',
    'юс малый': 'А',
    'юс большой': 'Ж'
}
print(old_letters.values())
# будет напечатан список начертаний старинных букв
# dict_values(['Ѣ', 'А', 'Ж'])
```

Методы словарей

Чтобы словари *превращать в строки*, используется метод *join()*:

```
# словарь отчеств
patronymic = {
    'Илья': 'Ильинична',
    'Иван': 'Ивановна',
    'Пётр': 'Петровна'
}
print(", ".join(patronymic.values()))
# выводится строка: Ильинична, Ивановна, Петровна
```

Методы словарей

clear()

Удаляет все элементы словаря.

```
>>> d2 = {'A1': '123', 'A2': '456'}
>>> print(d2)
{'A2': '456', 'A1': '123'}
>>> d2.clear()
>>> print(d2)
{}
```

Запуск в режиме
интерпретатора

Методы словарей

copy()

Создается новая копия словаря.

```
>>> d2 = { 'A1': '123', 'A2': '456' }
>>> d3 = d2.copy()
>>> print(d3)
{'A1': '123', 'A2': '456'}
>>> d3['A1'] = '789'
>>> print(d2)
{'A1': '123', 'A2': '456'}
>>> print(d3)
{'A1': '789', 'A2': '456'}
```

Методы словарей

update([other])

Обновить словарь парами (*key/value*) из *other*, если ключи уже существуют, то обновить их значения.

```
>>> d = { 'A1' : '123' , 'A2' : '456' }  
>>> d.update({ 'A1' : '333' , 'A3' : '789' })  
>>> print(d)  
{ 'A2' : '456' , 'A3' : '789' , 'A1' : '333' }
```

Расширение словаря

Расширение словаря можно сделать несколькими способами:

1) дописать новую пару в объявление словаря:

```
english = {  
    'рука': 'hand',  
    'нога': 'leg',  
    'бэкенд-разработчик': 'back-end developer',  
    'запрос': 'request' # новая пара ключ-значение }  

```

2) использовать доступ по ключу:

```
english['голова'] = 'head'  
# теперь в словаре есть запись 'голова': 'head'
```

Расширение словаря

При расширении словаря, из нескольких пар с одинаковыми ключами Python видит только одну — ту, что записана или добавлена последней:

```
my_wife = {  
    'жена': 'Оля',  
    'жена': 'Варя',  
    'жена': 'Вера Трифоновна'  
}  
print(my_wife)
```



Что получим?

Протокол вывода:

{'жена': 'Вера Трифоновна'}



В одном словаре не может быть двух одинаковых ключей.

Удаление элемента из словаря

Для удаления элемента из словаря можно воспользоваться командой *del*.

```
>>> d2 = {"A1": "123", "A2": "456"}  
>>> del d2["A1"]  
>>> print(d2)  
{ 'A2': '456' }
```

Перебор элементов словаря

Пройти по всем элементам словаря можно циклом `for`.

```
favorite_songs = {  
    'Тополиный пух': 'Иванушки international',  
    'Город золотой': 'Аквариум',  
    'Звезда по имени Солнце': 'Кино'  
}  
for track in favorite_songs:  
    print(track + ' это песня группы ' + favorite_songs[track])
```

Этот способ позволяет пробежать по всем ключам словаря. Обратите внимание, что `track` здесь — просто название переменной, оно могло быть любым и код отработал бы так же.

Перебор элементов словаря

Можно пройти отдельно по значениям словаря:

```
for music_band in favorite_songs.values():  
    print(' Я не могу слушать группу ' + music_band)
```

Можно пройти по ключам и значениям одновременно:

```
for track, music_band in favorite_songs.items():  
    print(track + ' это песня группы ' + music_band)
```

Здесь вызван метод `items()` — он похож на `keys()` и `values()`, но возвращает набор пар ключ-значение, поэтому при переборе мы используем две переменных — `track` и `music_band`. Их можно было назвать по-другому.

Программирование на языке Python

Множества

Множества



Множеством(set) в языке программирования Python называется неупорядоченная совокупность уникальных значений. В качестве элементов этого набора данных могут выступать любые неизменяемые объекты, такие как числа, символы, строки.

Множества:

- ✓ Дают возможность быстро удалять дубликаты, поскольку, по определению, могут содержать только уникальные элементы;
- ✓ Позволяют, в отличие от других коллекций, выполнять над собой ряд математических операций, таких как объединение, пересечение и разность множеств;

Множества

Пример set-ов в Python:

```
# множество натуральных чисел от 1 до 10
natural_num_set = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}

# множество персонажей Братства Кольца
the_fellowship = {'Гэндальф', 'Арагорн', 'Фродо',
                  'Сэм', 'Боромир', 'Леголас', 'Гимли'}

# множество приближений math.sqrt(2)
sqrt_set = {1.41421356, 1.4142135, 1.414213}

# множество результатов какого-то голосования
result_set = {('P', 76.69), ('G', 11.77),
              ('J', 5.65), ('S', 1.68), ('Y', 1.05) }
```

Создание множеств

Создать множество можно:

- 1) Присвоив переменной последовательность значений, выделив их фигурными скобками;

```
a = {1, 2, 0, 1, 3, 2}
print(a)
# Вывод
{0, 1, 2, 3}
```

- 2) Используя функцию **set**. Аргументом этой функции может быть набор неких данных или строка с текстом.

```
a = set('data')
print(a)
# Вывод
{'d', 'a', 't'}
```

Создание множеств

3) Для создания множества можно в Python воспользоваться генератором.

```
a = {i for i in [1, 2, 0, 1, 3, 2]}  
print(a)  
# Вывод  
{0, 1, 2, 3}
```



Замечание: пустое множество создаётся исключительно через `set()`

Использование множеств

Множество используется для следующих операций:

- 1) Проверка, есть ли данное значение в множестве. Для этого используется **in**:

```
a = {0, 1, 2, 3}
print(2 in a)
# Вывод
True
```

- 2) Проверка отсутствия данного значения в множестве. Используется **not in**:

```
a = {0, 1, 2, 3}
print(2 not in a)
# Вывод
False
```

Управления содержимым множеств

1) Получение размера множества:

Узнать точное количество элементов, входящих в состав множества, поможет метод `len`, принимающий в качестве аргумента набор данных.

```
a = {0, 1, 2, 3}
```

```
print(len(a))
```

```
# Вывод
```

```
4
```

Управления содержимым множеств

2) Сортировка множеств:

set list

{ 9 -1 3 } => [-1 3 9]

Операция сортировки отсутствует для множеств Python по определению. Множество — неупорядоченный набор. Но с помощью функции `sorted()`, можно получить отсортированный список:

```
some_digits = {1, 55, 34, 2, 12, 14, -4}
print(sorted(some_digits))
```

Вывод

```
[-4, 1, 2, 12, 14, 34, 55]
```

```
cities = {'Москва', 'Калининград', 'Белгород'}
print(sorted(cities))
```

Вывод

```
['Белгород', 'Калининград', 'Москва']
```

Управления содержимым множеств

3) Добавление элемента:

Чтобы внести новые значения, потребуется вызывать метод **add**. Аргументом в данном случае будет добавляемый элемент последовательности.

```
a = {0, 1, 2, 3}
```

```
a.add(4)
```

```
print(a)
```

```
# Вывод
```

```
{0, 1, 2, 3, 4}
```


Управления содержимым множеств

4) Добавление списка:

Для добавления нескольких элементов необходимо использовать метод **update**, который принимает итерабельный объект (список, кортеж, генератор и т.п.) и добавляет все входящие в него элементы в исходное множество.

```
my_set = {1, 2, 3}
example_list = ['q', 'w', 'e']
my_set.update(example_list)
print(my_set)
# Вывод
{1, 2, 3, 'q', 'e', 'w'}
```

Управления содержимым множеств

5) Удаление элемента:

Для удаления элементов из множества используются следующие функции в Python:

- ✓ `remove` — удаление элемента с генерацией исключения в случае, если такого элемента нет;
- ✓ `discard` — удаление элемента без генерации исключения, если элемент отсутствует;
- ✓ `pop` — удаление случайного элемента и возвращение его значения, генерируется исключение при попытке удаления из пустого множества.

```
A = {1, 2, 3, 4, 5}
A.discard(3)
A.remove(4)
print(A.pop())
print(A)
```

Протокол вывода:

```
1
{2, 5}
```

Управления содержимым множеств

6) Полная очистка:

Иногда необходимо полностью убрать все элементы. Чтобы не удалять каждый элемент отдельно, используется метод **clear**, не принимающий аргументов.

```
a = {0, 1, 2, 3}
a.clear()
print(a)
# Вывод
set()
```



В результате получили
пустое множество.

Управления содержимым множеств

7) Перебор элементов:

Множество, как и любую другую коллекцию, итерируем циклом for:

```
iterate_me = {1.1, 1.2, 1.3, 1.4, 1.5}
for num in iterate_me:
    print(num)
```

Вывод

1.1

1.4

1.3

1.2

1.5

Операции над множествами

Объединение множеств:

При использовании объединения множеств создаётся ещё одно множество, которое содержит все элементы, входящие в первоначальные два (без дубликатов).

Операция объединения в языке Python выполняется несколькими способами:

1) Чтобы объединить все элементы двух разных множеств, стоит воспользоваться методом **union** на одном из объектов.

```
a = {0, 1, 2, 3}
```

```
b = {4, 3, 2, 1}
```

```
c = a.union(b)
```

```
print(c)
```

```
# Вывод
```

```
{0, 1, 2, 3, 4}
```

Операции над множествами

2) Операция объединения в языке Python может быть выполнена, используя символ «|»

```
# используя символный метод
```

```
set_a = {1, 9, 22, 4}
```

```
set_b = {3, 4, 5, 6}
```

```
set_c = set_a | set_b
```

```
print(set_c)
```

```
# Вывод
```

```
{1, 3, 4, 5, 6, 9, 22}
```

Операции над множествами

Пересечение множеств:

При использовании пересечения множеств создаётся новый объект-множество, содержащий все элементы, общие для обоих (без дубликатов).

Операция пересечения выполняется несколькими способами:

1) Используя метод `intersection()`:

```
a = {0, 1, 2, 3}
b = {4, 3, 2, 1}
c = a.intersection(b)
print(c)
# Вывод
{1, 2, 3}
```

Операции над множествами

Пересечение множеств:

2) используя символ & :

```
# используя символьный метод
set_a = {1, 9, 22, 4}
set_b = {3, 4, 5, 6}
set_c = set_a & set_b
print(set_c)
# Вывод
{4}
```


Операции над множествами

Разность множеств:

При использовании разности множеств Питон создаёт новый объект, включающий элементы, которые есть в первом, но не входят во второй (в данном случае — в множестве `set_a`).

Операция разности выполняется несколькими способами:

1) Используя символ «-»:

```
# используя символьный метод
set_a = {1, 9, 22, 4}
set_b = {3, 4, 5, 6}
set_c = set_a - set_b
print(set_c)

# Вывод
{1, 9, 22}
```

Операции над множествами

Разность множеств:

2) Используя метод `difference()`:

```
# используя метод difference
set_a = {1, 9, 22, 4}
set_b = {3, 4, 5, 6}
set_c = set_a.difference(set_b)
print(set_c)

# Вывод
{1, 9, 22}
```

Операции над множествами

Симметричная разность множеств:

При применении симметричной разности на двух set-объектах создаётся новый объект, включающий все элементы, за исключением тех, которые есть в обоих.

Используется метод `symmetric_difference()`:

```
# используя метод symmetric_difference
set_a = {1, 9, 22, 4}
set_b = {3, 4, 5, 6}
set_c = set_a.symmetric_difference(set_b)
print(set_c)

# Вывод
{1, 3, 5, 6, 9, 22}
```

Операции над множествами

Метод `isdisjoint()` определяет, является ли пересечение двух множеств пустым:

```
it = {'green', 'white', 'red'}
ru = {'white', 'blue', 'red'}
ukr = {'blue', 'yellow'}
# вернет False, если пересечение множеств не
# пустое и True, в противном случае
print(ukr.isdisjoint(it))
print(ru.isdisjoint(it))
# Вывод
True
False
```



В Python нет оператора, который бы соответствовал этому методу.

Подмножество и надмножество в python

Подмножество: Множество `set_b` называется подмножеством `set_a`, если все элементы `set_b` принадлежат `set_a`.

Проверить, является ли множество `set_b` подмножеством `set_a` можно с помощью метода `issubset()`.

```
# используя метод issubset
set_a = {1, 9, 22, 4, 5}
set_b = {4, 1}
set_b.issubset(set_a)
# Вывод
True
```



Возвращает
True/False
исходя из результата
проверки.

Подмножество и надмножество в python

Надмножество: Множество `set_a` называется надмножеством `set_b`, если все элементы `set_b` входят в `set_a`.

Проверить на надмножество в Python можно используя метод `issuperset()` :

```
# используя метод issuperset
set_a = {1, 9, 22, 4, 5}
set_b = {22, 4}
set_a.issuperset(set_b)

# Вывод
True
```



Возвращает
True/False
исходя из результата
проверки.

Тип frozenset

Множество, содержимое которого не поддается изменению имеет тип frozenset. Значения из этого набора нельзя удалить, как и добавить новые.

```
a = frozenset({"hello", "world"})  
print(a)  
# Вывод  
frozenset({'hello', 'world'})
```



Поскольку содержимое frozenset должно всегда оставаться статичным, перечень функций, с которыми такое множество может взаимодействовать, имеет ограничения.

Тип frozenset

Над замороженными множествами можно производить операции:



- ☐ `union()` ;
- ☐ `intersection()` ;
- ☐ `difference()` ;
- ☐ `symmetric_difference()` ;

Методы изменяющие множество отсутствуют у frozenset:



- ☐ `add()`
- ☐ `remove()`
- ☐ `discard()`
- ☐ `pop()`
- ☐ `clear()`
- ☐ `update()`

 Результатом операций над замороженными множествами будут тоже замороженные множества.

Тип frozenset

Для создания замороженного множества используется встроенная функция `frozenset()`, которая принимает в качестве аргумента другую коллекцию.

Приведенный ниже код:

```
myset1 = frozenset({1, 2, 3})
myset2 = frozenset([1, 1, 2, 3, 4, 4, 4, 5, 6, 6])
myset3 = frozenset('aabcccddee')
print(myset1)
print(myset2)
print(myset3)
```

ВЫВОДИТ:

```
frozenset({1, 2, 3})
frozenset({1, 2, 3, 4, 5, 6})
frozenset({'e', 'd', 'c', 'b', 'a'})
```

Преобразование множеств

Строка

Для преобразования множества в строку используется конкатенация текстовых значений, которую обеспечивает метод `join`. Метод `type` возвращает тип данных объекта.

```
a = {'set', 'str', 'dict', 'list'}  
b = ','.join(a)  
print(b)  
print(type(b))  
# Вывод  
set,dict,list,str  
<class 'str'>
```

Преобразование множеств

Словарь

Чтобы получить из множества словарь, следует передать функции `dict` набор из нескольких пар значений, в каждом из которых будет находиться ключ.

```
a = { ('a', 2), ('b', 4) }
b = dict(a)
print(b)
print(type(b))
# Вывод
{'b': 4, 'a': 2}
<class 'dict'>
```



Каждый элемент для такого преобразования — кортеж состоящий из двух значений:

- ✓ ключ будущего словаря;
- ✓ значение, соответствующее ключу.

Преобразование множеств

Список

Можно получить список неких объектов. Для этого используется метод `list`, получающий в качестве аргумента множество `a`.

```
a = {1, 2, 0, 1, 3, 2}
```

```
b = list(a)
```

```
print(b)
```

```
print(type(b))
```

```
# Вывод
```

```
[0, 1, 2, 3]
```

```
<class 'list'>
```



Программирование на языке Python

Символьные строки

Символьные строки

Начальное значение:

```
s = "Привет!"
```



Строка – это
последовательность
символов!

Вывод на экран:

```
print ( s )
```

```
print ( s[5] )
```

```
print ( s[-2] )
```

0	1	2	3	4	5	6
П	р	и	в	е	т	!
s[0]	s[1]	s[2]	s[3]	s[4]	s[5]	s[6]

s[len(s)-2]

Длина строки:

```
n = len ( s )
```

Символьные строки

Ввод с клавиатуры:

```
s = input ( "Введите имя: " )
```

Изменение строки:

```
s[4]= "a"
```



Строка – это неизменяемый объект!

... но можно составить новую строку:

```
s1 = s + "a"
```


Символьные строки

Задача: заменить в строке все буквы "а" на буквы "б".

```
s = input ( "Введите строку: " )  
s1 = ""      # строка-результат  
for c in s:  
    if c == "а":  
        c = "б"  
    s1 = s1 + c  
print ( s1 )
```

перебрать все
символы в строке

добавить символ к
строке-результату

Стандартные функции и методы

Верхний/нижний регистр:

```
s = "aAbBcC"  
s1 = s.upper()    # "AABVCC"  
s2 = s.lower()    # "aabbcc"  
s3 = s.capitalize() # "Aabbcc"
```

Проверка на цифры:

```
s = "abc"  
print ( s.isdigit() )    # False  
s1 = "123"  
print ( s1.isdigit() )   # True
```

Стандартные функции и методы

Разделение строки:
метод – `split()`



Если в вызове `split()` не указывать разделитель, то строка разобьётся по пробелам

```
str = "a,b,c"  
print(str.split(","))  
#Выведет ['a', 'b', 'c']
```

Удаление лишних пробелов из строк

метод `strip()` - удаляет пробельные символы в начале и в конце строки;

`lstrip()` - удаляет пробельные символы в начале строки;

`rstrip()` - удаляет пробельные символы в конце строки.

Операции со строками

Объединение (конкатенация) :

```
s1 = "Привет"
```

```
s2 = "Петя"
```

```
s = s1 + ", " + s2 + "!"
```

"Привет, Петя!"

Срезы:

```
s = "0123456789"
```

```
s1 = s[3:8] # "34567"
```

0 1 2 3 4 5 6 7 8 9

0	1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

разрезы

0 1 2 3 4 5 6 7 8 9

Операции со строками

Срезы:

Срез `str[x:y]` позволит нам получить строку от символа `x` до `y`. При этом необязательно указывать оба параметра, ведь в таком случае срез произойдет от начала или до самого конца строки.

```
str = "Строки в Python и методы строк"  
print(str[9:15])  
# Выведет "Python"
```

Операции со строками

Срезы:

```
s = "0123456789"
```

```
s1 = s[:8] # "01234567"
```

от начала строки

```
s = "0123456789"
```

```
s1 = s[3:] # "3456789"
```

до конца строки

```
s1 = s[::-1] # "9876543210"
```

реверс строки

Операции со строками

```
s = "123456789"  
print(s[::2])          #"13579"
```

срезать каждый энный символ

Срезы с отрицательными индексами:

```
s = "0123456789"  
s1 = s[:-2]           # "01234567"
```

N-2

```
s = "0123456789"  
s1 = s[-6:-2]         # "4567"
```

N-6

N-2

Операции со строками

Удаление:

```
s = "0123456789"  
s1 = s[:3] + s[9:]    # "0129"  
      "012"      "9"
```

Вставка:

```
s = "0123456789"  
s1 = s[:3] + "ABC" + s[3:]  
      "012ABC3456789"
```


Форматирование строк

Для подстановки в строку любой переменной, которая может являться результатом работы основной части кода используются **f-строки**.

Применение *f*-строк позволяет включать значения переменных прямо в текстовую строку, обрамлённую кавычками.

В коде *f*-строки выглядят так же, как обычные строки, только перед открывающей кавычкой ставится символ *f*. Имена переменных в *f*-строках обрамляются в фигурные скобки.

Форматирование строк

Пример работы с f-строками:

```
num = 2*10  
num2 = 100  
s = f"Первое число: {num}. Второе число: {num2}"  
print(s)  
# Выведет "Первое число: 20. Второе число: 100"
```



Если переменная в f-строке имеет тип, отличный от `str`, то Python автоматически приведёт тип этой переменной к строке.

Экранирование специальных символов

`\\` — Обратный слеш
`\'` — Одинарная кавычка (апостроф)
`\"` — Двойная кавычка
`\b` — Backspace (BS)
`\n` — Новая строка (Line Feed, LF)
`\t` — Табуляция (TAB)

«Сырые» строки — это строки, в которых управляющие символы обрабатываются как есть, без специального значения:

```
# это обычная строка
regular_str = 'C:\\Windows\\cmd.exe'
# это "сырая" строка
raw_str = r'C:\Windows\cmd.exe'
```

Поиск в строках

Если нам нужно найти набор символов в начале или в конце данной строки, то используются следующие методы:

startswith()

Позволяет искать набор символов в начале строки.

```
s = "Строки в Python и методы строк"  
print(s.startswith("Строки"))  
#Выведет True
```

Поиск в строках

endswith()

Позволяет искать набор символов в конце строки.

```
s = "...я разбирал строки в JavaScript"  
print(s.endswith("JavaScript"))  
#Выведет True
```

Поиск в строках

Если нужно найти набор символов в произвольном месте строки, то используется метод `find()`.

```
s = "Здесь был Петя."  
n = s.find ( "с" )           # n = 3  
if n >= 0:  
    print ( "Номер символа", n )  
else:  
    print ( "Символ не найден." )
```



Находит первое слева вхождение подстроки!

Поиск с конца строки:

```
s = "Здесь был Вася."  
n = s.rfind ( "с" )           # n = 12
```

Проверки

`isupper()` — все символы в верхнем регистре?

`islower()` — все символы в нижнем регистре?

`istitle()` — первые символы всех слов в верхнем регистре, а остальные в нижнем?

`isalpha()` — все символы — буквы латинского алфавита?

`isspace()` — все символы пробелы?

`isdecimal()` — все символы десятичные цифры?

`isnumeric()` — все символы числовые (включая индексы, степени и дробные символы) ?

Замена в строках

Для замены одного набора символов на другой в строке используют метод **replace()**.

```
str = "Строки в Python и методы"  
str2=str.replace("Python", "JavaScript")  
print(str2)  
#Выведет  
#Строки в JavaScript и методы
```


Пример обработки строк

Задача: Ввести имя, отчество и фамилию. Преобразовать их к формату «фамилия-инициалы».

Пример:

Введите имя, отчество и фамилию:

Василий Алибабаевич Хрюндиков

Результат:

Хрюндиков В.А.

Алибабаевич Хрюндиков

Хрюндиков

Алгоритм:

- найти первый пробел и выделить имя
- удалить имя с пробелом из основной строки
- найти первый пробел и выделить отчество
- удалить отчество с пробелом из основной строки
- «сцепить» фамилию, первые буквы имени и фамилии, точки, пробелы...

Хрюндиков В.А.

Вариант 1

```
print ( "Введите имя, отчество и фамилию:" )
s = input()
n = s.find ( " " )
name = s[:n]      # вырезать имя
s = s[n+1:]
n = s.find ( " " )
name2 = s[:n]      # вырезать отчество
s = s[n+1:]        # осталась фамилия
s = s + " " + name[0] + "." + name2[0] + "."
print ( s )
```

Вариант 2

```
print ( "Введите имя, отчество и фамилию:" )  
s = input()  
fio = s.split()  
s = fio[2] + " " + fio[0][0] + "." + fio[1][0] + ". "  
print ( s )
```

Василий	Алибабаевич	Хрюндиков
fio[0]	fio[1]	fio[2]

Преобразования «строка» – «число»

Из строки в число:

```
s = "123"
N = int ( s )           # N = 123
s = "123.456"
X = float ( s )         # X = 123.456
```

Из числа в строку:

```
N = 123
s = str ( N )           # s = "123"
s = "{:5d}".format(N)   # s = "  123"

X = 123.456
s = str ( X )           # s = "123.456"
s = "{:7.2f}".format(X) # s = " 123.46"
s = "{:10.2e}".format(X) # s = " 1.23e+02"
```



Программирование на языке Python

**Массивы. Обработка
элементов. Сортировка.**

Как обработать все элементы массива?

Обработка с переменной:

```
i = 0;  
# обработать A[i]  
i += 1  
# обработать A[i]  
i += 1  
# обработать A[i]  
i += 1  
# обработать A[i]  
i += 1  
# обработать A[i]  
i += 1
```



Обработка в цикле:

```
i = 0  
while i < N:  
    # обработать A[i]  
    i += 1
```



Цикл с переменной:

```
for i in range(N):  
    # обработать A[i]
```

Ввод массива с клавиатуры

Создание массива:

```
N = 5  
A = [0] * N
```

Ввод с клавиатуры:

```
for i in range(N):  
    print ( "A[" , i , "]" = "  
            sep = " ", end = " " )  
    A[i] = int( input() )
```

A[0] = 5
A[1] = 12
A[2] = 34
A[3] = 56
A[4] = 13

sep = "
end = "

не разделять
элементы

не переходить на
новую строку

Ввод массива с клавиатуры

Ввод без подсказок:

```
A = [ int(input()) for i in range(N) ]
```

Ввод в одной строке:

```
data = input()      # "1 2 3 4 5"  
s = data.split()    # ["1", "2", "3", "4", "5"]  
A = [ int(x) for x in s ]  
                    # [1, 2, 3, 4, 5]
```

или так:

```
s = input().split() # ["1", "2", "3", "4", "5"]  
A = list( map(int, s) ) # [1, 2, 3, 4, 5]
```

построить
список

применить `int` ко
всем элементам `s`

Вывод массива на экран

Как список:

```
print ( A )
```

 [1, 2, 3, 4, 5]

В строчку через пробел:

```
for i in range(N):  
    print ( A[i], end=" " )
```

 1 2 3 4 5

или так:

```
for x in A:  
    print ( x, end=" " )
```

 1 2 3 4 5

или так:

```
print ( *A )
```



```
print (1, 2, 3, 4, 5)
```

Заполнение случайными числами

```
from random import randint
N = 10
A = [0]*N
for i in range(N):
    A[i] = randint(20,100)
```

или так:

```
from random import randint
N = 10
A = [ randint(20,100)
      for x in range(N) ]
```

случайные
числа
[20,100]

Перебор элементов

Общая схема (можно изменять $A[i]$):

```
for i in range(N):  
    ... # сделать что-то с A[i]
```

```
for i in range(N):  
    A[i] += 1
```

Если не нужно изменять $A[i]$:

```
for x in A:  
    ... # сделать что-то с x
```

$x = A[0], A[1], \dots, A[N-1]$

```
for x in A:  
    print ( x )
```

Подсчёт нужных элементов

Задача. В массиве *A* записаны данные о росте баскетболистов. Сколько из них имеет рост больше 180 см, но меньше 190 см?



Как решать?

Python:

`180 < x < 190`

```
count = 0
for x in A:
    if 180 < x and x < 190:
        count += 1
```

Перебор элементов

Сумма:

```
summa = 0
for x in A:
    if 180 < x < 190:
        summa += x
print ( summa )
```

или так:

```
print ( sum(A) )
```

Перебор элементов

Среднее арифметическое:

```
count = 0
summa = 0
for x in A:
    if 180 < x < 190:
        count += 1
        summa += x
print ( summa/count )
```

среднее
арифметическое

или так:

```
B = [ x for x in A
      if 180 < x < 190 ]
print ( sum(B)/len(B) )
```

отбираем нужные

Программирование на языке Python

Алгоритмы обработки массивов

Поиск в массиве

```
for i in range ( N ) :  
    if A[i] == X :  
        print ( "A[" , i , "]" = " , X , sep = " " )  
        break  
else :  
    print ( "Не нашли!" )
```

```
if X in A :  
    nX = A.index (X)  
    print ( "A[" , nX , "]" = " , X , sep = " " )  
else :  
    print ( "Не нашли!" )
```

Максимальный элемент

```
M = A[0]
for i in range(1, N):
    if A[i] > M:
        M = A[i]
print ( M )
```



Если `range(N)` ?

Варианты в стиле Python:

```
M = A[0]
for x in A:
    if x > M:
        M = x
```

```
M = max ( A )
```

Максимальный элемент и его номер

```
M = A[0]; nMax = 0
for i in range(1, N):
    if A[i] > M:
        M = A[i]
        nMax = i
print( "A[" , nMax, "]=", M, sep = " " )
```



Что можно улучшить?



По номеру элемента можно найти значение!

```
nMax = 0
for i in range(1, N):
    if A[i] > A[nMax]:
        nMax = i
print( "A[" , nMax, "]=", A[nMax], sep = " " )
```

Максимальный элемент и его номер

Вариант в стиле Python:

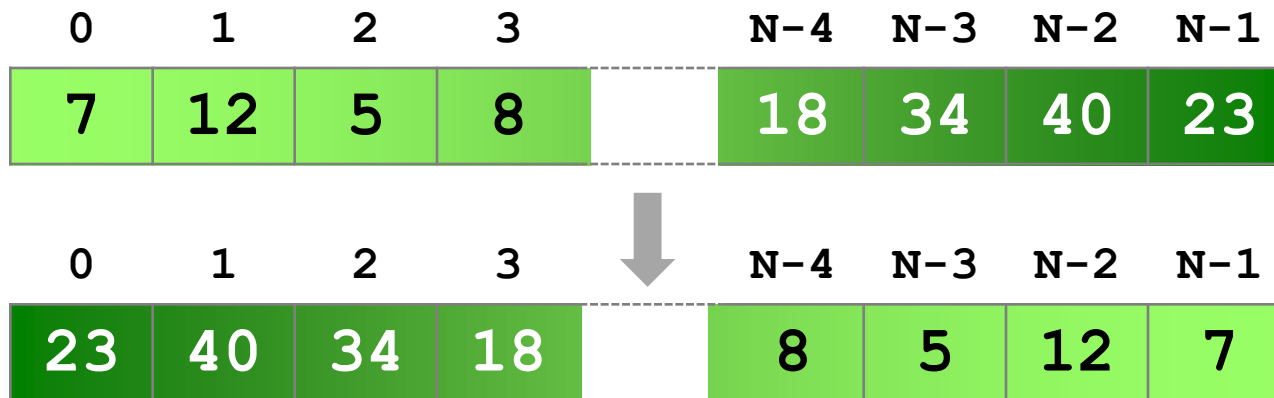
```
M = max(A)
nMax = A.index(M)
print ( "A[" , nMax , "]" = " , M , sep = " " )
```

номер заданного
элемента (первого из...)



Для поиска минимального элемента и его номера используются аналогичные программы с применением функции **min**

Реверс массива



«Простое» решение:

остановиться на середине!

```
for i in range( N//2 ):
    поменять местами A[i] и A[N-1-i]
```



Что плохо?

Реверс массива

```
for i in range(N//2):  
    c = A[i]  
    A[i] = A[N-1-i]  
    A[N-1-i] = c
```

Варианты в стиле Python:

```
for i in range(N//2):  
    A[i], A[N-i-1] = A[N-i-1], A[i]
```

```
A.reverse()
```

Циклический сдвиг элементов

0	1	2	3		N-4	N-3	N-2	N-1
7	12	5	8		18	34	40	23

↓

0	1	2	3		N-4	N-3	N-2	N-1
12	5	8	15		34	40	23	7

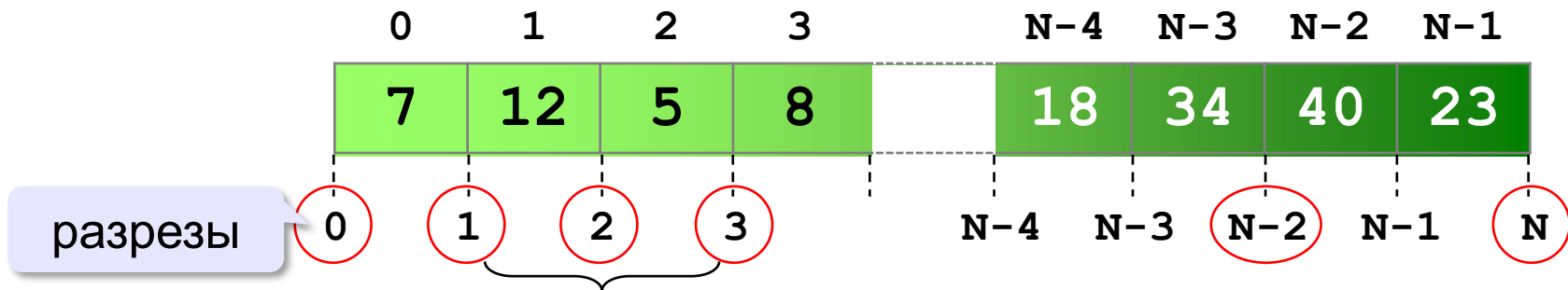
«Простое» решение:

```
for i in range(N-1):
    A[i] = A[i+1]
```



Почему не до N?

Срезы в Python



`A[1:3]` → `[12, 5]`

`A[2:3]` → `[5]`

`A[:3]` → `A[0:3]` → `[7, 12, 5]`

с начала

`A[3:N-2]` → `[8, ..., 18, 34]`

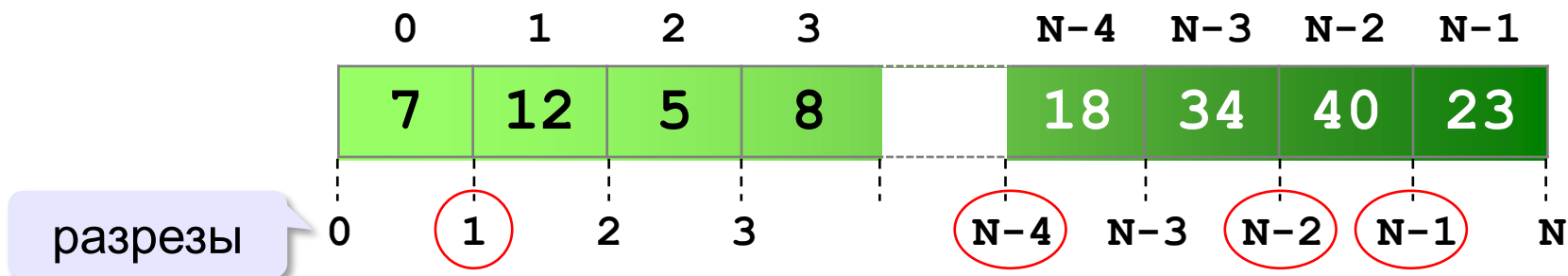
`A[3:]` → `A[3:N]` → `[8, ..., 18, 34, 40, 23]`

до конца

копия массива

`A[:]` → `[7, 12, 5, 8, ..., 18, 34, 40, 23]`

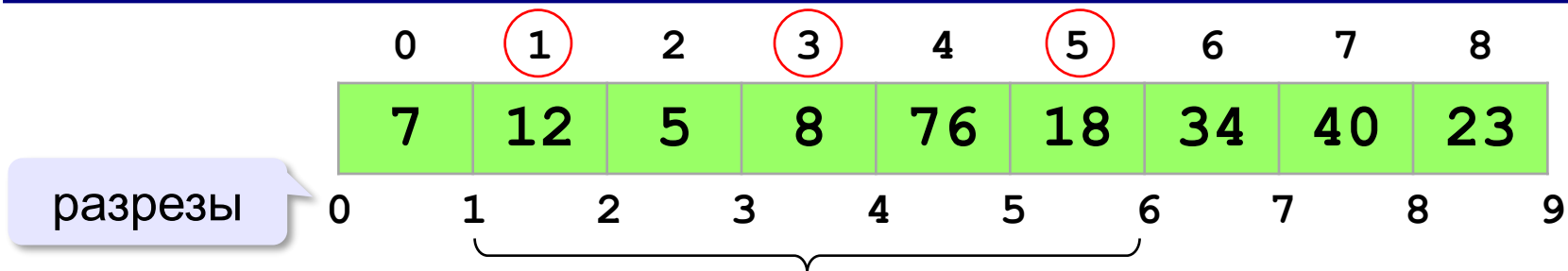
Срезы в Python – отрицательные индексы



$A[1:-1]$
 $A[1:N-1]$ $\rightarrow$ $[12, 5, 8, \dots, 18, 34, 40]$

$A[-4:-2]$
 $A[N-4:N-2]$ $\rightarrow$ $[18, 34]$

Срезы в Python – шаг



`A[1:6:2]` → [12, 8, 18]

`A[: :3]` → [7, 8, 34]

`A[8:2:-2]` → [23, 34, 76]

`A[: :-1]` → [23, 40, 34, 18, 76, 8, 5, 12, 7]

реверс!

`A.reverse()`

Отбор нужных элементов

Задача. Отобрать элементы массива **A**,
удовлетворяющие некоторому условию, в массив **B**.

Простое решение:

```
B = []  
сделать для i от 0 до N-1  
    если условие выполняется для A[i] то  
        добавить A[i] к массиву B
```

```
B = []  
for x in A:  
    if x % 2 == 0:  
        B.append(x)
```



Какие элементы выбираем?

добавить **x** в конец
массива **B**

Отбор нужных элементов

Задача. Отобрать элементы массива **A**,
удовлетворяющие некоторому условию, в массив **B**.

Решение в стиле Python:

перебрать все
элементы A

```
B = [ x for x in A  
      if x % 2 == 0 ]
```

если **x** – чётное
число

Особенности работы со списками

$A = [1, 2, 3]$

$B = A$

$A \rightarrow [1, 2, 3]$
 $B \rightarrow [1, 2, 3]$



$A[0] = 0$

$A \rightarrow [0, 2, 3]$
 $B \rightarrow [0, 2, 3]$

$A = [1, 2, 3]$

$B = A[:]$

копия массива A

$A \rightarrow [1, 2, 3]$

$B \rightarrow [1, 2, 3]$



$A[0] = 0$

$A \rightarrow [0, 2, 3]$

$B \rightarrow [1, 2, 3]$

Программирование на языке Python

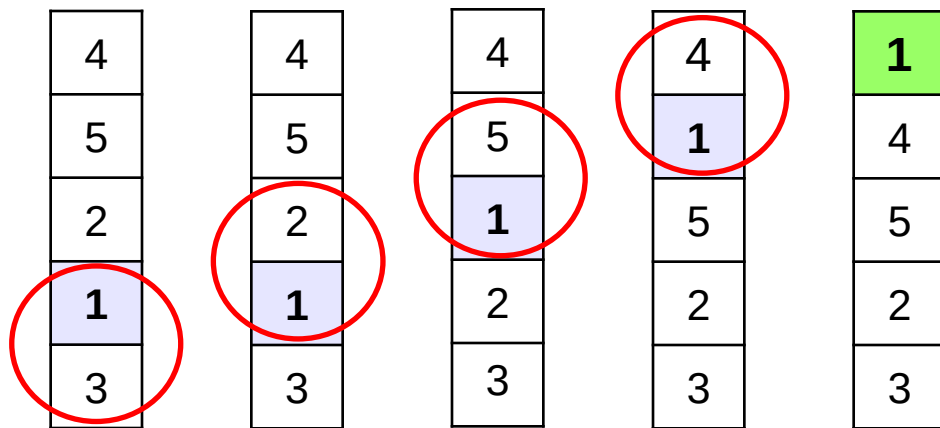
Сортировка

Метод пузырька (сортировка обменами)

Идея: пузырек воздуха в стакане воды поднимается со дна вверх.

Для массивов – **самый маленький** («легкий» элемент перемещается вверх («всплывает»)).

1-й проход:



- сравниваем два соседних элемента; если они стоят «неправильно», меняем их местами
- за 1 проход по массиву **один** элемент (самый маленький) становится на свое место

Метод пузырька

2-й проход:

1	1	1	1
4	4	4	2
5	5	2	4
2	2	5	5
3	3	3	3

3-й проход:

1	1	1
2	2	2
4	4	3
5	3	4
3	5	5

4-й проход:

1	1
2	2
3	3
4	4
5	5



Для сортировки массива из N элементов нужен $N-1$ проход (достаточно поставить на свои места $N-1$ элементов).

Метод пузырька

1-й проход:

```
сделать для j от N-2 до 0 шаг -1  
    если A[j+1] < A[j] то  
        # поменять местами A[j] и A[j+1]
```

2-й проход:

```
сделать для j от N-2 до 1 шаг -1  
    если A[j+1] < A[j] то  
        # поменять местами A[j] и A[j+1]
```

единственное
отличие!

Метод пузырька

от $N-2$ до 0 шаг -1

1-й проход:

```
for j in range(N-2, -1, -1):  
    if A[j+1] < A[j]:  
        # поменять местами A[j] и A[j+1]
```

единственное
отличие!

2-й проход:

```
for j in range(N-2, 0, -1):  
    if A[j+1] < A[j]:  
        # поменять местами A[j] и A[j+1]
```

Метод пузырька

```
for i in range(N-1):  
    for j in range(N-2, i-1, -1):  
        if A[j+1] < A[j]:  
            A[j], A[j+1] = A[j+1], A[j]
```

Метод выбора (минимального элемента)

Идея: найти минимальный элемент и поставить его на первое место.

```
for i in range(N-1):  
    найти номер nMin минимального  
        элемента из A[i]..A[N]  
    if i != nMin:  
        поменять местами A[i] и A[nMin]
```

Метод выбора (минимального элемента)

```
for i in range(N-1):  
    nMin = i  
    for j in range(i+1, N):  
        if A[j] < A[nMin]:  
            nMin = j  
    if i != nMin:  
        A[i], A[nMin] = A[nMin], A[i]
```



Программирование на языке Python

Матрицы

Что такое матрица?

	○	×
	○	×
○	×	

нет знака

A

НОЛИК

крестик

строка 1,
столбец 2

A[1][2]

?

Как закодировать?

Матрица — это прямоугольная таблица, составленная из элементов одного типа (чисел, строк и т.д.). Каждый элемент матрицы имеет два индекса — номер строки и номер столбца.

Создание матриц

! Матрица – это список списков!

```
A = [[-1, 0, 1],  
      [-1, 0, 1],  
      [0, 1, -1]]
```

перенос на другую
строку внутри скобок

или так:

```
A = [[-1, 0, 1], [-1, 0, 1], [0, 1, -1]]
```

! Нумерация элементов с нуля!

A[0][0]	A[0][1]	A[0][2]
A[1][0]	A[1][1]	A[1][2]
A[2][0]	A[2][1]	A[2][2]

Создание матриц

Нулевая матрица:

```
A = []  
for i in range(N):  
    A.append( [0]*M )
```

Количество строк

Добавление элемента в список

Количество элементов в строке

Например:

```
A = []  
for i in range(3):  
    A.append( [0]*2 )
```

Результат:

0	0
0	0
0	0

Ввод элементов матрицы с клавиатуры:

После создания нулевой матрицы размера $N \times M$,
заполняем ее элементами введенными с клавиатуры:

Количество строк

Количество элементов в
строке

```
for i in range(N):  
    for j in range(M):  
        A[i][j]= int(input())
```

Вывод матриц

```
print ( A )
```

```
[[1, 12, 3], [4, 5, 146], [7, 118, 99]]
```

```
def printMatrix ( A ):
```

```
    for row in A:
```

```
        for x in row:
```

```
            print ( "{:4d}".format(x) , end = "" )
```

```
        print ()
```

```
1   12   3
4    5 146
7 118   99
```



Зачем форматный вывод?

Простые алгоритмы

Заполнение случайными числами:

```
import random
for i in range(N):
    for j in range(M):
        A[i][j] = random.randint( 20, 80 )
        print ( "{:4d}".format(A[i][j]),
                end = " " )
    print()
```



Вложенный цикл!

Суммирование:

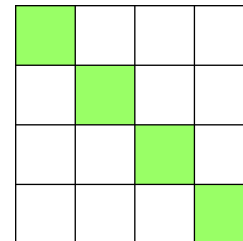
```
s = 0
for i in range(N):
    for j in range(M):
        s += A[i][j]
print ( s )
```

```
s = 0
for row in A:
    s += sum(row)
print ( s )
```

Перебор элементов матрицы

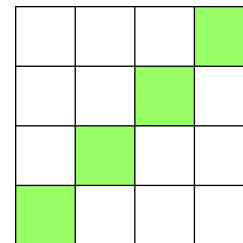
Главная диагональ:

```
for i in range(N):  
    # работаем с A[i][i]
```



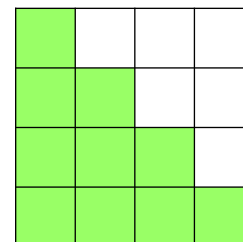
Побочная диагональ:

```
for i in range(N):  
    # работаем с A[i][N-1-i]
```



Главная диагональ и под ней:

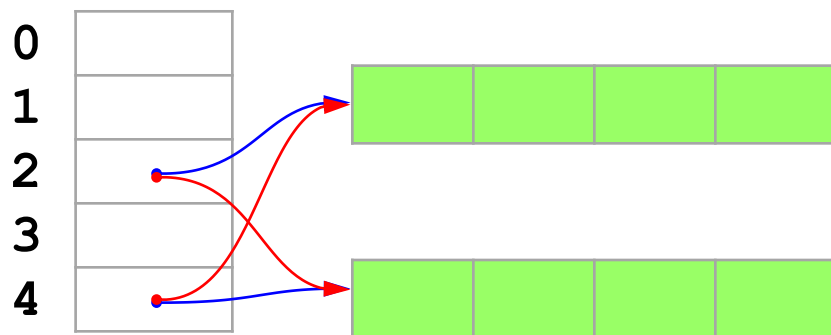
```
for i in range(N):  
    for j in range(i+1):  
        # работаем с A[i][j]
```



Перестановка строк и столбцов

2-я и 4-я строки:

```
A[2], A[4] = A[4], A[2]
```



2-й и 4-й столбцы:

```
for i in range(N):  
    A[i][2], A[i][4] = A[i][4], A[i][2]
```

Выделение строк и столбцов

1-я строка:

```
R = A[1][:]
```

```
R = A[i]
```

2-й столбец:

```
C = []  
for row in A:  
    C.append(row[2])
```

или так:

```
C = [ row[2] for row in A ]
```

главная диагональ:

```
D = [ A[i][i] for i in range(N) ]
```


Сортировка в Python

По возрастанию:

```
B = sorted( A )
```

алгоритм
Timsort

По убыванию:

```
B = sorted( A, reverse = True )
```

По последней цифре:

```
def lastDigit ( n ):  
    return n % 10  
B = sorted( A, key = lastDigit )
```

или так:

```
B = sorted( A, key = lambda x: x % 10 )
```

«лямбда»-функция
(функция без имени)

Сортировка в Python – на месте

По возрастанию:

```
A.sort()
```

По убыванию:

```
A.sort ( reverse = True )
```

По последней цифре:

```
def lastDigit ( n ):  
    return n % 10  
A.sort ( key = lastDigit )
```

или так:

```
A.sort ( key = lambda x: x % 10 )
```

Программирование на языке Python

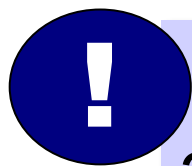
Списки.

Вложенные списки.

Списки

Список (list) — это одномерный динамический массив. То есть, это массив в который можно добавлять и убирать элементы по ходу работы программы:

```
# создадим пустой список
my_list = []
# добавим в список новый элемент
my_list.append(5)
# удалим из списка элемент
my_list.remove(5)
```

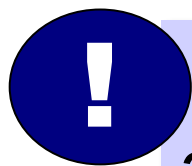


Список помнит порядок следования элементов!

Списки

Список предоставляет доступ к значениям по индексам (номерам, которые отсчет начинаются с нуля):

```
my_list = [3, 1, 4, 1]
# создали список с 4 элементами
my_list[0] = 2
# заменяем первый элемент списка
print(my_list)
# печатает [2, 1, 4, 1]
```



Элементами списка могут быть значения разных типов!

Способы создания списков

Создание пустого списка:

```
my_list_1 = []
```

На базе нескольких существующих элементов:

```
my_list_2 = [False, 1, 2.0, 'c']
```

На базе коллекции (строка, диапазон и т.п.):

```
my_list_4 = list('Hello')  
# ['H', 'e', 'l', 'l', 'o']  
my_list_5 = list(range(10))  
# [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Проверка на присутствие или отсутствие элемента в списке

```
my_list = [False, 1, 2.0, 'c', 'elem']  
  
if 'elem' in my_list:  
    print('Элемент принадлежит списку')  
  
if 'elem' not in my_list:  
    print('Элемент не принадлежит списку')
```

Результат:
Элемент принадлежит списку



Что получим?

Добавление элемента в список

Добавление элемента в конец списка:

```
my_list = [ 1, 2.0, 'c']  
my_list.append('new elem')  
print(my_list)      #[1, 2.0, 'c', 'new elem']
```

Вставка элемента в середину списка:

```
my_list = [ 1, 2.0, 'c']  
my_list.insert(2, 'third elem')  
print(my_list)      #[1, 2.0, 'third elem', 'c']
```

Добавление элемента в список

Добавление в список всех элементов из другого списка:

```
my_list = [ 1, 2.0, 'c']  
another_list = [ 'f', 5.0, True]  
my_list.extend(another_list)  
print(my_list)  #[1, 2.0, 'c', 'f', 5.0, True]
```

Создание нового списка в результате слияния (конкатенации) двух списков:

```
list1  = [ 1, 2.0, 'c']  
list2  = [ 1, 2.0, 'f']  
new_list = list1 + list2  
print(new_list)  #[1, 2.0, 'c', 1, 2.0, 'f']
```

Поиск элемента в списке

Поиск индекса (номера) первого вхождения элемента в список (если элемент отсутствует — будет ошибка `ValueError`):

```
i = my_list.index(elem)
```

Подсчёт количества повторов элемента в списке:

```
num = my_list.count(elem)
```

Удаление элементов из списка

Удалить элемент из списка по индексу (номеру):

```
del my_list[0]
```

Удалить элемент из списка по значению (если элемент отсутствует — будет ошибка `ValueError`):

```
my_list.remove('elem')
```

Очистка списка:

```
my_list.clear()
```

Извлечение элемента из списка

Извлечение элемента из конца списка (элемент удаляется из списка):

```
last_element = my_list.pop()
```

Извлечение элемента из середины списка (элемент удаляется из списка):

```
third_element = my_list.pop(2)
```

Встроенные функции и методы:

Сумма элементов списка:

```
sum(my_list)
```

Максимальный элемент списка:

```
max(my_list)
```

Минимальный элемент списка:

```
min(my_list)
```

Сортировка списка (in-place):

```
my_list.sort()
```

Сортировка списка в
убывающем порядке (in-place):

```
my_list.sort(reverse=True)
```

Создание отсортированной
копии списка:

```
new_list = sorted(my_list)
```

Встроенные функции и методы:

При сортировке можно указать параметр **key**, который позволяет задать функцию преобразования элементов при их сравнении. Отметим, что элементы списка при этом остаются прежними:

Например, таким образом можно отсортировать строки по их длине, а не значению:

```
words = ['abc', 'bc', 'c']  
words.sort(key=len)  
# перед сравнением строк будет вычисляться  
# их длина  
print(words) # ['c', 'bc', 'abc']
```

Вложенные списки

Если элементами списков являются другие списки, то такие списки называются **вложенными списками**.

Элементами вложенного списка могут быть любые типы данных – числа, строки, булевы значения и другие списки...

Например:

```
numbers = [10, 3, [3.1415, 2.71828, 1.1415],  
            ['Russia', 'Armenia',  
            'Argentina', [True, False]]]
```


Типы вложенных списков:

- Регулярный список списков.
- Нерегулярный список списков.

1) Каждый элемент в регулярном списке списков называется подсписком, таким образом соблюдается единообразие типа элемента.

Например: `[[9, 8, 7], [6, 5, 4], [3, 2, 1]]` – имеет тип `list`.

2) Каждый элемент в нерегулярном списке списков является либо подсписком, либо элементом, не являющимся списком. Таким образом, возникает нестыковка по типу элемента.

Например: `[[9, 8, 7], [6, 5], 4, 3]` – это нерегулярный список списков, поскольку `[9, 8, 7]` и `[6, 5]` относятся к типам `list`, тогда как `4` и `3` имеют тип `int`.

Создание вложенного списка

Чтобы создать вложенный список, перечисляем элементы через запятую в квадратных скобках:

Например:

```
my_list = [[0], [1, 2], [3, 4, 5]]
```

```
print(my_list[0])  
print(my_list[1])  
print(my_list[2])  
print(len(my_list))
```

Результат:

```
[0]  
[1, 2]  
[3, 4, 5]  
3
```

Считывание вложенных списков

Если элементы списка вводятся через клавиатуру (каждая строка на отдельной строке, всего n строк, числа в строке разделяются пробелами), для ввода списка можно использовать следующий код:

```
n = 4 # количество строк (элементов)
my_list = []
for i in range(n):
    elem = [int(i) for i in input().split()]
my_list.append(elem)
```

то в переменной `my_list` будет храниться список:
[[2, 4], [6, 7, 8, 9], [1, 3], [5, 6, 5, 4, 3, 1]]

В результате, если на вход программе подаются строки:

```
2 4
6 7 8 9
1 3
5 6 5 4 3 1
```

Индексация вложенного списка

Так как элементы вложенного списка – строки и списки, их можно индексировать.

Рассмотрим программный код:

```
my_list = ['Python', [10, 20, 30],  
           ['Beegreek', 'Stepik!']]  
print(my_list[0][2])  
print(my_list[1][1])  
print(my_list[2][-1])  
print(my_list[2][-1][-1])  
print(my_list[3])
```

Результат:

t

20

Stepik!

!

IndexError: index
out of range

Функция len()

Рассмотрим программный код:

```
my_list = [[0], [1, 2], [3, 4, 5], [],  
           [10, 20, 30]]  
print(len(my_list))
```

Результат:
5

! Функция len() возвращает количество элементов (число 5) списка my_list, а не общее количество элементов во всех списках (число 9).

? Как получить общее кол-во элементов во всех вложенных списках?

Функция len()

Если требуется посчитать общее количество элементов во вложенном списке, необходимо использовать цикл for в связке с функцией len():

Рассмотрим программный код:

```
total = 0
my_list = [[0], [1, 2], [3, 4, 5], [], [10,
                20, 30]]
for li in my_list:
    total += len(li)
print(total)
```

Результат:

9

Функции `max()`, `min()`

Функции `min()` и `max()` также можно использовать при работе с вложенными списками.

Рассмотрим программный код:

```
list1 = [[1, 7, 12, 0, 9, 100], [1, 7, 90],  
         [1, 10]]  
list2 = [['c', 'b'], ['d'], ['a', 'p', 'q']]  
print(min(list1))  
print(max(list1))  
print(min(list2))  
print(max(list2))
```

Результат:

[1, 7, 12, 0, 9, 100]

[1, 10]

['a', 'p', 'q']

['d']



элементы вложенных
списков должны быть
сравнимы!

Функции max(), min()

Рассмотрим программный код:

```
my_list = [[1, 7, 12, 0, 9, 100], ['a', 'b']]  
print(min(my_list))  
print(max(my_list))
```

Результат:

TypeError: '<' not supported between
instances of 'str' and 'int'

Примечание 1.

Независимо от вложенности списков, используются все списочные методы:

1. метод `append()` добавляет новый элемент в конец списка.
2. метод `extend()` расширяет один список другим списком.
3. метод `insert()` вставляет значение в список в заданной позиции.
4. метод `index()` возвращает индекс первого элемента, значение которого равняется переданному в метод значению.
5. метод `remove()` удаляет первый элемент, значение которого равняется переданному в метод значению.

Примечание 1.

6. метод `pop()` удаляет элемент по указанному индексу и возвращает его.
7. метод `count()` возвращает количество элементов в списке, значения которых равны переданному в метод значению.
8. метод `reverse()` инвертирует порядок следования значений в списке, то есть меняет его на противоположный.
9. метод `copy()` создает поверхностную копию списка.
10. метод `clear()` удаляет все элементы из списка.
11. оператор `del` позволяет удалять элементы списка по определенному индексу.

Примечание 2.

Методы строк, работающие со списками:

1. метод `split()` разбивает строку на слова, через разделитель (в качестве разделителя могут быть: последовательность пробельных символов, символ табуляции (`\t`) или символ новой строки (`\n`)).
2. метод `join()` собирает строку из элементов списка.

Примечание 3.

Язык Python не ограничивает нас в уровнях вложенности: элементами списка могут быть списки, их элементами могут быть другие списки, элементами которых в свою очередь могут быть другие списки...

Обработка вложенных списков

Для обработки элементов вложенного списка, так же как и для вывода его элементов на экран используются **вложенные циклы**.

Используем два вложенных цикла для подсчета суммы всех чисел в списке:

```
my_list = [[1, 9, 8, 7, 4], [7, 3, 4], [2, 1]]
total = 0
for i in range(len(my_list)):
    for j in range(len(my_list[i])):
        total += my_list[i][j]
print(total)
```

Обработка вложенных списков

Используем два вложенных цикла для подсчета суммы всех чисел в списке не по индексу, а по значениям:

```
my_list = [[1, 9, 8, 7, 4], [7, 3, 4], [2, 1]]
total = 0
for row in my_list:
    for elem in row:
        total += elem
print(total)
```

Пример. Упаковка дубликатов

На вход программе подается строка текста, содержащая символы, отделенные символом пробела. Программа должна вывести указанный вложенный список.

Формат ввода:

g i v e t h h i i s m a a a n a g u u n

Формат вывода:

[['g'], ['i'], ['v'], ['e'], ['t'], ['h' , 'h'], ['i' , 'i'], ['s'], ['m'], ['a' , 'a' , 'a'], ['n'],
['a'], ['g'], ['u' , 'u'], ['n']]

Пример. Упаковка дубликатов

```
lst1=[]
lst2=[]
for char in input().split():
    if lst1==[]:
        lst1.append(char)
    elif lst1[-1]==char:
        lst1.append(char)
    else:
        lst2.append(lst1)
        lst1=[]
        lst1.append(char)
if lst1:
    lst2.append(lst1)
print(lst2)
```


Программирование на языке Python

Кортежи.

Кортежи

В Python имеются **неизменяемые** последовательности содержащие, в отличие от строк, абсолютно произвольные данные. Такие коллекции называются **кортежами**.

Тип данных кортежа - **tuple**

Рассмотрим программный код:

```
my_list = [1, 2, 3, 4, 5] # список
```

```
my_tuple = (1, 2, 3, 4, 5)
```

```
# Заменяв квадратные скобки при объявлении
```

```
# списка на круглые, мы объявили кортеж
```



Кортежи по своей природе (задумке) – **неизменяемые аналоги списков.**

Кортежи

Рассмотрим программный код:

```
my_tuple = (1, 2, 3, 4, 5)
my_tuple[0] = 9
my_tuple[4] = 7
print(my_tuple)
```

?

Результат работы программы?

Результат:

`TypeError: 'tuple' object does not support item assignment`

!

Кортеж (tuple) – ещё один вид коллекций в Python. Похож на список, но, в отличие от списка, **неизменяемый**.

Примеры кортежей

```
empty_tuple = ()  
# пустой кортеж  
point = (1.5, 6.0)  
# кортеж из двух чисел  
names = ('Timur', 'Ruslan', 'Roman')  
# кортеж из трех строк  
info = ('Timur', 'Guev', 28, 170, 60, False)  
# кортеж из 6 элементов разных типов  
nested_tuple = (('one', 'two'), ['three', 'four'])  
# кортеж из кортежа и списка
```



Кортежи могут хранить и содержать в себе объекты любых типов (даже составных) и поддерживают неограниченное количество уровней вложенности.

Кортеж с одним элементом

Для создания кортежа с единственным элементом после значения элемента ставят закрывающую запятую:

```
my_tuple = (1,)  
print(type(my_tuple)) # <class 'tuple'>
```

Если запятую пропустить, то кортеж создан не будет. Например, приведенный ниже код просто присваивает переменной `my_tuple` целочисленное значение 1:

```
my_tuple = (1)  
print(type(my_tuple)) # <class 'int'>
```

Свойства кортежей

Неизменяемость кортежей обеспечивает им особые свойства:

Скорость – кортежи быстрее работают, так как из-за неизменяемости хранятся в памяти иначе, и операции с их элементами выполняются заведомо быстрее, чем с компонентами списка.

Безопасность – неизменяемость превращает их в идеальные константы. Заданные кортежами константы делают код более читаемым и безопасным. Кроме того, в кортеже можно безопасно хранить данные, не опасаясь, что они будут случайно или преднамеренно изменены в программе.

Примечание

Кортеж является неизменяемым, но мы можем поменять содержимое списка в кортеже.

```
my_tuple = (1, 'python', [1, 2, 3])  
print(my_tuple)  
my_tuple[2][0] = 100  
my_tuple[2].append(17)  
print(my_tuple)
```

Результат:

(1, 'python', [1, 2, 3])

(1, 'python', [100, 2, 3, 17])



Меняется список, а не кортеж. Списки являются ссылочными типами данных, поэтому в кортеже хранится ссылка на список, которая не меняется при изменении самого списка.

Функция tuple()

Встроенная функция tuple() может применяться для преобразования **списка в кортеж**.

```
str_list = ['один', 'два', 'три']  
str_tuple = tuple(str_list)  
print(str_tuple)
```

Выводит:
('один' , 'два' , 'три')

Аналогичным образом мы можем создать кортеж на основании строки.

```
text = 'hello python'  
str_tuple = tuple(text)  
print(str_tuple)
```



Символ пробел
содержится в
кортеже str_tuple

Выводит:
('h' , 'e' , 'l' , 'l' , 'o' , ' ' , 'p' , 'y' , 't' , 'h' , 'o' , 'n')

Преобразование кортежа в список

```
writer = ('Лев Толстой', 1827)
print(writer)
a = list(writer)
a[1] = 1828
writer = tuple(a)
print(writer)
```

Выводит:

```
('Лев Толстой', 1827)
('Лев Толстой', 1828)
```



Может быть полезно, когда нужно изменить элементы кортежа, например для устранения ошибок

Особенности кортежей

Кортежи поддерживают те же операции, что и списки, за исключением изменяющих содержимое.

Кортежи поддерживают:

- доступ к элементу по индексу (только для получения значений элементов);
- методы, в частности `index()`, `count()`;
- встроенные функции, в частности `len()`, `sum()`, `min()` и `max()`;
- срезы;
- оператор принадлежности `in`;
- операторы конкатенации (+) и повторения (*).

Кортежи не поддерживают такие методы, как `append()`, `remove()`, `pop()`, `insert()`, `reverse()`, `sort()`, так как они изменяют содержимое.

Оператор принадлежности in

Оператор in позволяет проверить, содержит ли кортеж некоторый элемент.

```
numbers = (2, 4, 6, 8, 10)
if 2 in numbers:
    print('Кортеж содержит число 2')
else:
    print('Кортеж не содержит число 2')
```

Выводит:
Кортеж содержит число 2



Можно использовать оператор in вместе с логическим оператором not

Метод count()

Метод count() возвращает количество элементов в кортеже, значения которых равны переданному в метод значению.

```
names = ('Timur', 'Gvido', 'Roman',  
         'Timur', 'Anders', 'Timur')  
cnt1 = names.count('Timur')  
cnt2 = names.count('Gvido')  
cnt3 = names.count('Josef')  
print(cnt1)  
print(cnt2)  
print(cnt3)
```

Выводит:

3
1
0

Метод partition()

Метод partition() принимает на вход один аргумент sep, разделяет строку при первом появлении sep и **возвращает кортеж**, состоящий из трех элементов: часть перед разделителем, сам разделитель и часть после разделителя. Если разделитель не найден, то кортеж содержит саму строку, за которой следуют две пустые строки.

```
s1 = 'abc-de'.partition('-')
s2 = 'abc-de'.partition('.')
s3 = 'abc-de-fgh'.partition('-')
print(s1)
print(s2)
print(s3)
```

Выводит:

```
('abc', '-', 'de')
('abc-de', '', '')
('abc', '-', 'de-fgh')
```

Вложенные кортежи

Можно создавать вложенные кортежи.

```
colors = ('red', ('green', 'blue'), 'yellow')
numbers = (1, 2, (4, (6, 7, 8, 9)), 10, 11)
print(colors[1][1])
print(numbers[2][1][3])
```

Выводит:
blue
9

Упаковка кортежей

Упаковкой кортежа называют присваивание его какой-либо переменной.

```
tuple1 = (1, 2, 3)
tuple2 = ('b',)
tuple3 = ('red', 'green', 'blue', 'cyan')
print(type(tuple1))
print(type(tuple2))
print(type(tuple3))
```

Выводит:

```
<class 'tuple'>
<class 'tuple'>
<class 'tuple'>
```



Упаковка выполняется всегда, когда справа от знака равенства стоит больше одного значения.

Распаковка кортежей

Обратная операция, смысл которой в том, чтобы присвоить значения элементов кортежа отдельным переменным называется **распаковкой кортежа**

```
colors = ('red', 'green', 'blue', 'cyan')
(a, b, c, d) = colors
print(a)
print(b)
print(c)
print(d)
```

Выводит:
red
green
blue
cyan



можем опустить скобки слева от знака равенства:
a, b, c, d = colors

Распаковка кортежей

Количество переменных должно совпадать с числом элементов в кортеже.

```
colors = ('red', 'green', 'blue', 'cyan')  
a, b = colors
```

Выводит:
ValueError: too many values to unpack

```
colors = ('red', 'green', 'blue')  
a, b, c, d = colors
```

Выводит:
ValueError: not enough values to unpack (expected 4, got 3)

Распаковка кортежей

Если необходимо получить лишь какие-то отдельные значения, то в качестве "ненужных" переменных позволено использовать символ нижнего подчеркивания _

```
colors = ('red', 'green', 'blue')  
a, b, _ = colors  
print(a)  
print(b)
```

Выводит:
red
green

Распаковка кортежей

Распаковка кортежей очень удобна, когда необходимо менять местами значения переменных без использования временных переменных.

```
a, b, c = 3, 2, 1  
b, a, c = c, a, b  
print(b, c, a)
```

Выводит:
1 2 3

Распаковка кортежей

Если при распаковке кортежа число элементов слева и справа не совпадает, то возникает ошибка времени исполнения. Есть способ собрать сразу **несколько значений в одну переменную**. Это делается при помощи звездочки перед именем переменной.

```
a, b, *tail = 1, 2, 3, 4, 5, 6  
print(a)  
print(b)  
print(tail)
```

Выводит:

```
1  
2  
[3, 4, 5, 6]
```



Переменная помеченная * всегда будет списком, даже в нее попадает лишь один элемент или даже ноль.

Распаковка кортежей

Звездочка может быть только у одного аргумента, но необязательно у последнего.

```
singer = ('Freddie', 'Bohemian Rhapsody',  
          'Killer Queen', 'Love of my life',  
          'Mercury')  
name, *songs, surname = singer  
print(name)  
print(songs)  
print(surname)
```

ВЫВОДИТ:

Freddie

['Bohemian Rhapsody', 'Killer Queen', 'Love of my life']

Mercury

